

Name: Hasini Kotha
Reg No: 24011101057
Class: AI-DS A

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Source Code

Google Colab Notebook:

<https://colab.research.google.com/drive/1FkWNyt7JhQYDpUTc7-uIIInVzhY7qL8a-?usp=sharing>

GitHub Repository:

https://github.com/Hasini-Kotha/Deep_Learning

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector

- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

7.1 Feature Histograms

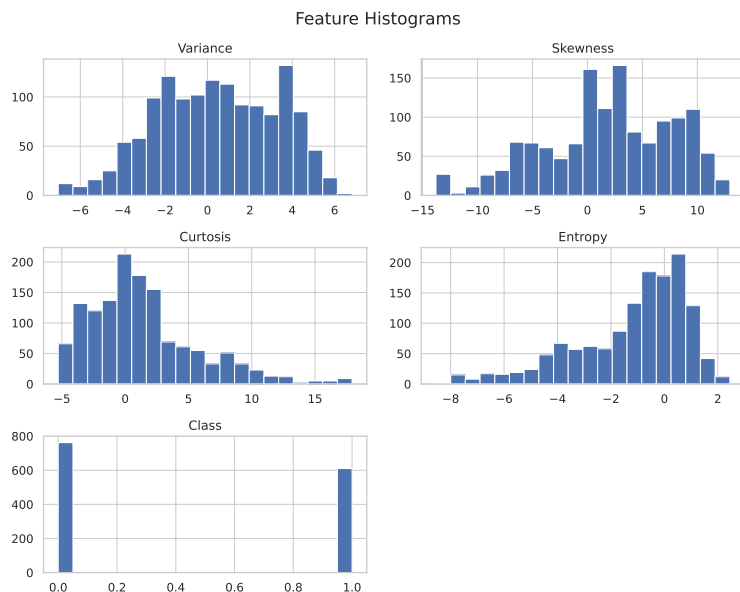


Figure 1: Feature Histograms - Distribution Analysis

Description: The histograms show that each feature has a different distribution, with some features spread over a wider range than others. A few extreme values (outliers) are present, but overall the data appears suitable for analysis. The class distribution is fairly balanced, indicating that both authentic and forged banknotes are well represented in the dataset.

7.2 Correlation Heatmap

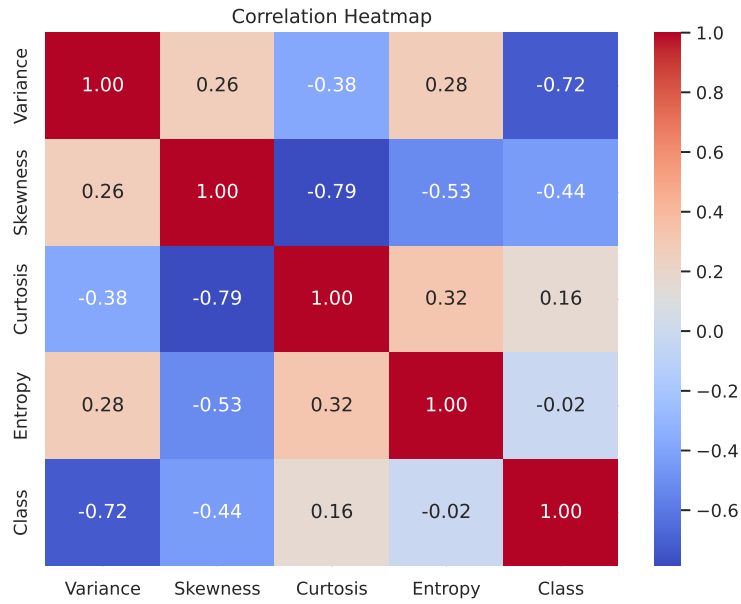


Figure 2: Correlation Heatmap - Feature Relationship

Description: The correlation heatmap shows the strength and direction of the relationship between the features. Variance has a strong negative correlation with the target class (-0.72), making it one of the most influential features for classification, while Entropy has almost no correlation with the class (-0.02). Overall, the features have varying levels of correlation, indicating that each contributes different information to the perceptron model.

7.3 Scatter Plot

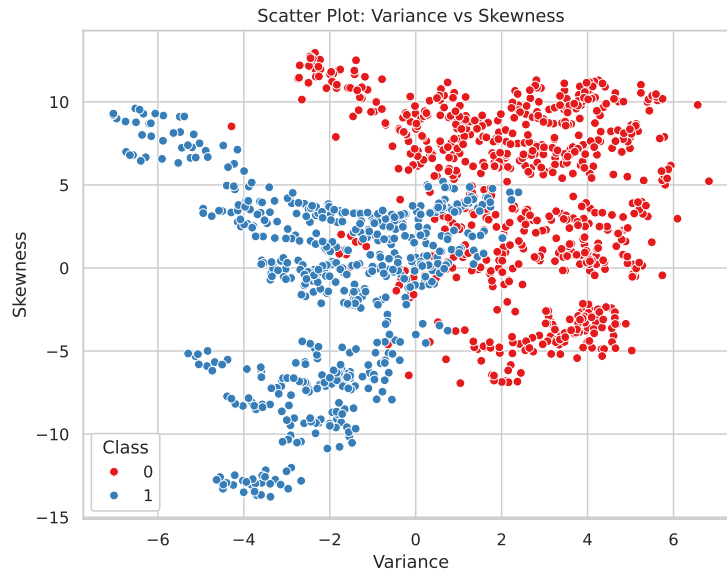


Figure 3: Scatter Plot - Class Distribution

Description: The scatter plot shows that the two classes form distinct clusters with only a small overlapping region. This indicates that the data is approximately linearly separable using the features Variance and Skewness, making it suitable for classification using a Single Layer Perceptron.

7.4 Training Error vs Epoch

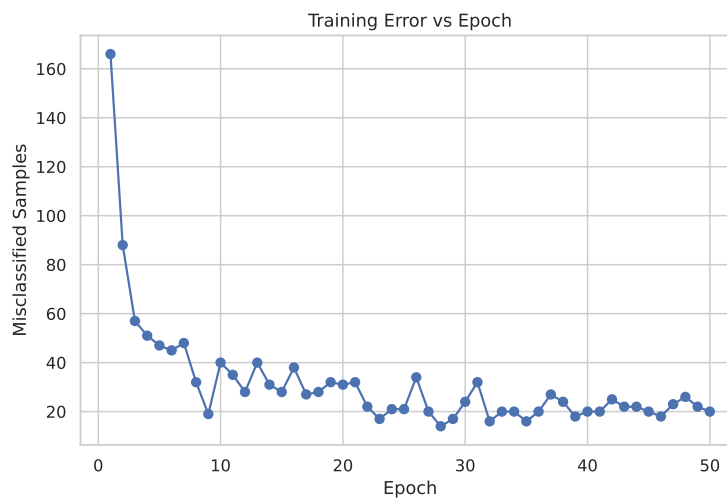


Figure 4: Training Error vs Epoch - Learning Behaviour

Description: The training error decreases significantly during the early epochs, indicating that the perceptron is successfully learning from the training data. As training progresses, the error levels off, suggesting that the model has reached a stable solution and additional epochs do not produce substantial improvements.

7.5 Weight Evolution

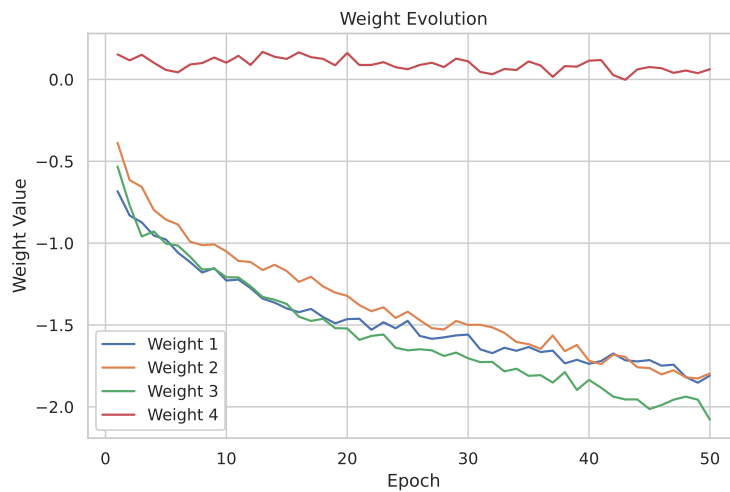


Figure 5: Weight Evolution - Parameter Analysis

Description: The weight values are updated in every epoch based on the perceptron learning rule. As training progresses, the changes become smaller and the weights stabilize, showing that the model has learned the importance of each feature and reached convergence.

7.6 Bias Evolution

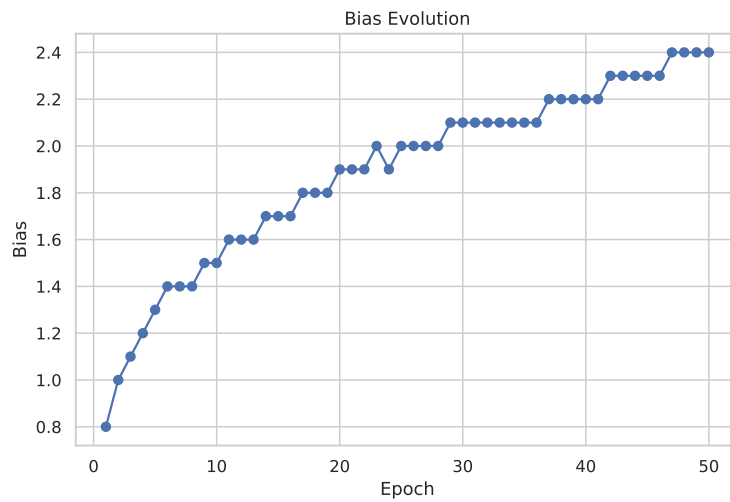


Figure 6: Bias Evolution - Parameter Analysis

Description: The graph shows how the bias is updated during each epoch of training. The gradual stabilization of the bias value indicates that the perceptron has learned an appropriate decision threshold and that further training results in minimal changes.

7.7 Confusion Matrix

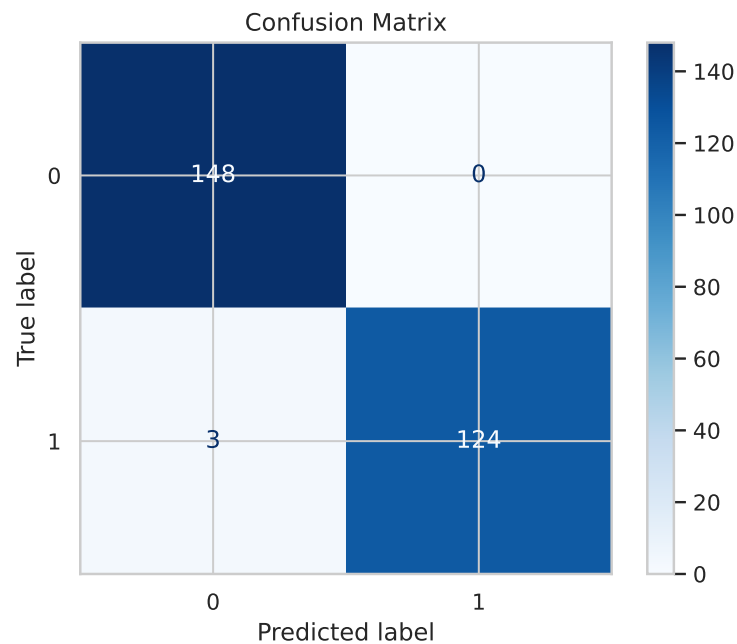


Figure 7: Confusion Matrix - Performance Evaluation

Description: The confusion matrix indicates that the model correctly classified almost all banknotes, with only 3 misclassifications out of 275 test samples. This demonstrates that the perceptron achieved high accuracy and performed well in distinguishing between authentic and forged banknotes.

7.8 Learning Rate Comparison

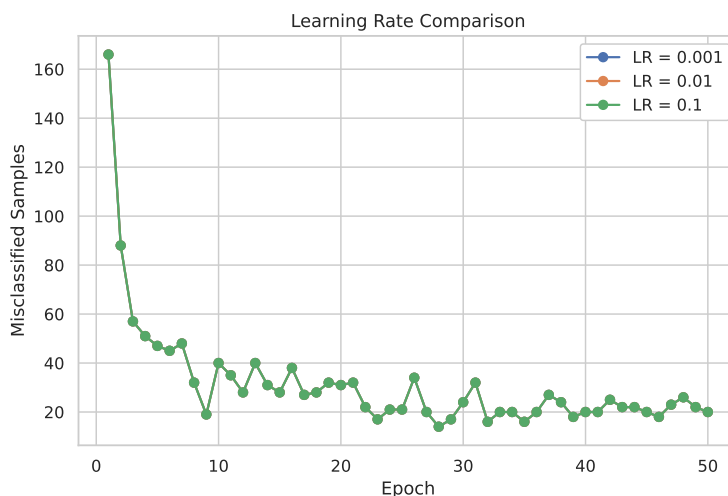


Figure 8: Learning Rate Comparison - Hyperparameter Study

Description: The learning rate comparison shows that all three learning rates produced nearly identical training error curves. This suggests that changing the learning rate had little effect on the perceptron's convergence, and all three learning rates achieved similar performance on this dataset.

8. Performance Tables

Training Summary

Metric	Value
Dataset Size	1372 Samples
Train/Test Split	80% Training / 20% Testing (1097 / 275)
Learning Rate	0.01 (Best Performing Learning Rate)
Epochs	50 (Best Training Error at Epoch 28: 14 Misclassified Samples)
Final Weights	[-0.1809, -0.1796, -0.2079, 0.0062]
Final Bias	0.24
Accuracy	98.91%
Precision	100.00%
Recall	97.64%
F1-score	98.80%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	166	-0.6827	-0.3867	0.8
2	88	-0.8312	-0.6146	1.0
3	57	-0.8732	-0.6566	1.1
4	51	-0.9539	-0.7974	1.2
5	47	-0.9778	-0.8564	1.3
6	45	-1.0581	-0.8862	1.4
7	48	-1.1160	-0.9913	1.4
8	32	-1.1796	-1.0124	1.4
9	19	-1.1535	-1.0082	1.5
10	40	-1.2286	-1.0505	1.5
11	35	-1.2228	-1.1087	1.6
12	28	-1.2748	-1.1154	1.6
13	40	-1.3390	-1.1646	1.6
14	31	-1.3637	-1.1323	1.7
15	28	-1.3991	-1.1707	1.7
16	38	-1.4218	-1.2369	1.7
17	27	-1.4019	-1.2055	1.8
18	28	-1.4513	-1.2647	1.8
19	32	-1.4900	-1.3019	1.8
20	31	-1.4641	-1.3221	1.9
21	32	-1.4620	-1.3786	1.9
22	22	-1.5294	-1.4160	1.9
23	17	-1.4834	-1.3919	2.0
24	21	-1.5205	-1.4572	1.9
25	21	-1.4746	-1.4186	2.0
26	34	-1.5666	-1.4704	2.0
27	20	-1.5845	-1.5194	2.0
28	14	-1.5757	-1.5276	2.0
50	20	-1.8092	-1.7965	2.4

9. Conclusion

The experiment successfully demonstrated the implementation of a Single Layer Perceptron from scratch for binary classification using the Banknote Authentication dataset. The results showed

that a perceptron with the Step Activation Function can effectively classify linearly separable data with high accuracy. Data preprocessing, including feature normalization, contributed to efficient learning and improved model performance.

The trained model achieved an accuracy of 98.91%, along with high precision, recall, and F1-score, indicating reliable classification of authentic and forged banknotes. The comparison of different learning rates showed that all three learning rates (0.001, 0.01, and 0.1) produced similar convergence behaviour and classification performance on this dataset. Overall, the experiment provided a clear understanding of the perceptron learning algorithm, parameter updates, and the role of activation functions in binary classification.

10. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.
4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

10.1 Comparison of Step and Sigmoid Activation Functions

The Step activation function outputs discrete values (0 or 1), resulting in a derivative that is zero everywhere except at the threshold where it is undefined. Modern deep learning models rely on gradient-based optimization algorithms (like backpropagation), which require differentiable activation functions with informative, non-zero gradients to update weights. The Sigmoid function, by contrast, is a smooth, continuous curve that maps inputs to a range between 0 and 1. Its derivative is well-defined everywhere, enabling effective error propagation and weight adjustments in modern deep neural networks.

10.2 Comparison with Scikit-learn's Perceptron

A comparison was made between the custom Perceptron implemented from scratch and the built-in `Perceptron` from Scikit-learn. Both models were trained using a learning rate of 0.01 and 50 epochs.

Metric	From Scratch	Scikit-learn
Accuracy	98.91%	98.55%
Precision	100.00%	100.00%
Recall	97.64%	96.85%
F1-score	98.80%	98.40%
Final Bias	0.24	0.24

Final Weights:

- **From Scratch:** $[-0.1809, -0.1796, -0.2079, 0.0062]$
- **Scikit-learn:** $[-0.1884, -0.1850, -0.1991, 0.0077]$

Inference: Both models performed exceptionally well on the Banknote Authentication dataset, achieving over 98.5% accuracy. The custom implementation slightly outperformed the Scikit-learn model, likely due to internal differences in data shuffling, tie-breaking, or optimization strategies used by the library. However, the resulting weights are highly similar, and the final bias values are identical (0.24), confirming the correctness and efficiency of the from-scratch implementation.

10.3 Effect of Different Learning Rates

As evidenced by the Learning Rate Comparison plot and training summaries, testing the perceptron with learning rates of 0.001, 0.01, and 0.1 yielded very similar convergence behaviours. Because the Banknote Authentication dataset is largely linearly separable, the perceptron easily identifies a valid decision boundary. Consequently, varying the learning rate within this reasonable range did not drastically alter the number of epochs required to converge or the final accuracy.

10.4 The XOR Problem and Single Layer Perceptrons

A Single Layer Perceptron is fundamentally a linear classifier; it can only draw a single straight line (or hyperplane) to separate classes. The XOR problem represents a non-linearly separable dataset—there is no single straight line that can separate the inputs that yield 1 from those that yield 0. Therefore, a Single Layer Perceptron cannot solve the XOR problem. Complex, non-linear problems like this necessitate Multi-Layer Perceptrons (MLPs) with hidden layers and non-linear activation functions.

10.5 Effect of Feature Normalization on Convergence

Feature normalization ensures that all input variables are scaled to a similar range. In the Banknote Authentication dataset, features have different numerical scales. Without normalization, features with larger magnitudes would disproportionately influence weight updates, leading to a skewed loss landscape, erratic learning paths, and much slower convergence. Normalizing the features ensures a more symmetric error surface, which enables the perceptron to converge faster and more stably.